



HowTo: RMI

☰ Tags	OP
# Cap	-99
⚙ Status	Done
🕒 Ultima modifica	@February 11, 2024 3:16 PM

▼ Quick Links

[Architettura](#)

[Errori e consigli dei prof](#)

[Passi per l'utilizzo](#)

[Compilazione & esecuzione](#)

[Lato server](#)

[Lato client](#)

[Documentazione sull'RMI \(Remote Method Invocation\) in Java](#)

[Introduzione a RMI](#)

[Concetti Chiave di RMI](#)

[1. Oggetti Remote](#)

[2. Registro RMI](#)

[3. Stubs e Skeletons](#)

[4. Serializzazione](#)

[Implementazione di RMI](#)

[1. Definire l'interfaccia Remota](#)

[2. Implementare l'interfaccia Remota](#)

[3. Creare e avviare il Registro RMI](#)

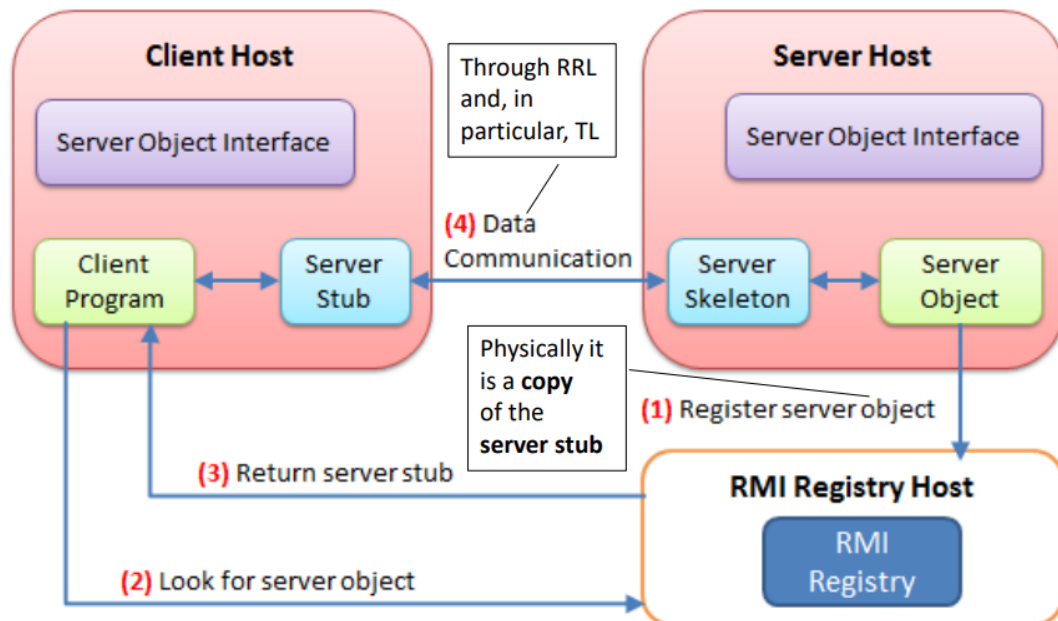
[4. Creare il Client RMI](#)

[Compilazione e Esecuzione](#)

[Conclusioni](#)

▼ Architettura

▼ Schema



RMI Registry

- Serve da DNS
- I server ci si registrano
- I client gli richiedono gli indirizzi dei server

Interfaccia

- Interfaccia che serve sia al server che al client

Server

- Implementazione dell'interfaccia
- Si registra all'RMI Registry

Skeleton

Parte del server generata, serve per interfacciarsi come RMI

Stub

Parte del server generata, da mandare al client (deve averla per RMI)

Client

- Chiede informazioni al RMI Registry

- Richiama metodi dell'interfaccia ed eseguiti sul server

Errori e consigli dei prof

- Controllare sempre argomenti in input
- Le funzioni che accedono allo stato condiviso devono essere synchronized

Passi per l'utilizzo

1. Definire **interfaccia** ed **implementazione** del server
2. Compilare l'interfaccia e la classe con `javac` e poi generare stub e skeleton, con `rmic`
3. Pubblicare il servizio nel sistema di nomi, registry o RMIRegistry
 - a. Attivare il registry
 - b. Il server deve fare una bind sul registry
4. Ottenere lato cliente il riferimento all'oggetto tramite name service facendo lookup sul registry

Compilazione & esecuzione

Lato server

1. Compilazione interfaccia ed implementazione

```
javac EchoInterface.java EchoRMIServer.java
```

2. Generazione eseguibili Stub & Skeleton

```
rmic -vcompat EchoRMIServer
```

3. Esecuzione lato server

- a. `rmiregistry`
- b. `java EchoRMIServer`

Lato client

1. Compilazione `javac EchoRMIClient.java`
 2. Esecuzione `java EchoRMIClient`
-

Documentazione sull'RMI (Remote Method Invocation) in Java

Introduzione a RMI

RMI, acronimo di Remote Method Invocation, è una tecnologia di programmazione che consente a un'applicazione Java di invocare i metodi di oggetti che risiedono su un'altra JVM (Java Virtual Machine), possibilmente su una macchina remota. RMI permette di creare applicazioni distribuite in Java, in cui gli oggetti possono comunicare e invocare metodi su oggetti remoti come se fossero locali.

Concetti Chiave di RMI

1. Oggetti Remote

Gli oggetti che possono essere invocati da remoto devono implementare l'interfaccia `java.rmi.Remote` e dichiarare che lanciano l'eccezione `java.rmi.RemoteException`.

2. Registro RMI

Il registro RMI è un servizio che mappa nomi di oggetti a riferimenti remoti. Gli oggetti remoti devono essere registrati nel registro RMI prima che possano essere invocati da client remoti.

3. Stubs e Skeletons

Quando un oggetto remoto viene esportato, il sistema RMI crea automaticamente un "stub" (sostituto) e un "skeleton" (scheletro) per l'oggetto.

Il client usa il stub per invocare i metodi sull'oggetto remoto, mentre il server usa lo scheletro per ricevere le richieste dal client e inoltrarle all'oggetto remoto.

4. Serializzazione

Poiché i parametri e i valori restituiti dai metodi remoti devono essere trasferiti attraverso la rete, devono essere serializzabili. Java fornisce il meccanismo di serializzazione per trasformare gli oggetti Java in una sequenza di byte che può essere inviata attraverso la rete.

Implementazione di RMI

1. Definire l'interfaccia Remota

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface MyRemoteInterface extends Remote {  
    public String sayHello() throws RemoteException;  
}
```

2. Implementare l'interfaccia Remota

```
import java.rmi.RemoteException;  
import java.rmi.server.UnicastRemoteObject;  
  
public class MyRemoteObject extends UnicastRemoteObject implements MyRemoteInterface {  
    public MyRemoteObject() throws RemoteException {  
        super();  
    }  
  
    public String sayHello() throws RemoteException {  
        return "Hello, world!";  
    }  
}
```

3. Creare e avviare il Registro RMI

```
import java.rmi.registry LocateRegistry;
import java.rmi.registry Registry;

public class MyServer {
    public static void main(String[] args) {
        try {
            MyRemoteInterface remoteObject = new MyRemoteObject();
            Registry registry = LocateRegistry.createRegistry(1099);
            registry.rebind("MyRemoteObject", remoteObject);

            System.out.println("Server ready");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

4. Creare il Client RMI

```
import java.rmi.registry LocateRegistry;
import java.rmi.registry Registry;

public class MyClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            MyRemoteInterface remoteObject = (MyRemoteInterface) registry.lookup("MyRemoteObject");
            String response = remoteObject.sayHello();
            System.out.println("Response from server: " + response);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
}  
}  
}
```

Compilazione e Esecuzione

1. Compilare le classi Java con il comando `javac`.
2. Avviare il registro RMI con il comando `rmiregistry`.
3. Avviare il server RMI.
4. Avviare il client RMI.

Conclusioni

RMI è uno strumento potente per la creazione di applicazioni distribuite in Java. Con RMI, è possibile trasferire oggetti e invocare metodi in modo trasparente su macchine remote, facilitando lo sviluppo di sistemi distribuiti robusti e scalabili. La documentazione sopra fornisce una panoramica dei concetti fondamentali e dell'implementazione pratica di RMI in Java.